

数字逻辑设计

林世俊

linsj@xmu.edu.cn

18950091347, QQ群: 910875018

Xiamen University





课程简介

- 随着工艺技术的发展，数字电路的规模越来越大，基于小规模芯片的数字系统设计方法的应用范围已经越来越小，取而代之的是基于编程语言和可编程器件的数字系统设计方法。
- 从目前市场上来看，**FPGA**的容量越来越大，功能越来越强大，价格越来越便宜，开发的门槛以及成本越来越低，因此应用也越来越广泛。在产品的销售规模不大的情况下，**FPGA**已经可以做到比**ASIC**更加节约成本；即使是销售规模很大的产品，在开发的第一阶段，一般都还是需要在**FPGA**上进行原型验证。可以说，**FPGA**已经是整个信息产业领域不可缺少的一个器件。



课程简介

- 对于一个本科生来说，如果具备基于**FPGA**开发大规模数字系统的能力，无论是对于今后的科研或者工作都是非常重要。因此，该课程以**FPGA**为核心，并且基于现有的实验平台，拟达到的教学目的和任务如下：
 - 第一，让学生掌握最常用的大规模数学电路设计流程和设计方法；
 - 第二，熟悉开发工具、掌握调试方法；
 - 第三，精通硬件编程语言。



课程简介

- 教学方法：理论讲授和实验相结合。
- 教学的内容：

章	主要内容
1	数字电路主要设计方法和流程
2	Verilog语言、基本组合逻辑电路的设计
3	Verilog语言、基本时序逻辑电路的设计
4	Verilog语言、基于状态机的数字电路设计
5	综合编程训练

课程简介

➤ 参考资料:

➤ Verilog HDL设计实用教程, 黄海 于斌主编, 清华大学出版社

➤ PPT

➤ 要求

➤ 每一个已完成的设计均需要找我验收登记

➤ 成绩构成: 考勤**10%**+所登记设计的难度和数目**80%**+实验报告**10%**

实验报告提交

- 从已完成的实验中挑选一个最满意的实验，撰写实验报告，报告内容包括实验内容介绍、设计思路、设计代码、仿真结果和下载板子结果五个部分。在课程结束后一周内，提交**word**文件（电子版），按照“实验名称+名字+学号”命名



目录

- 可编程器件
- 开发平台及开发软件
- 大规模数字电路设计流程
- **verilog**语言
- 实验内容推荐

可编程器件

➤ CPLD

烧写，断电程序不丢失
容量小

➤ FPGA

断电程序消失，需要PROM辅助

容量大

❖ 可配置逻辑模块CLB（
Configurable Logic Block）

❖ 输入、输出模块I/OB

❖ 可编程互联总线PI（Programmable Interconnect）

开发平台



FPGA: Xilinx公司的Artix-7 系列XC7A75T , 主要相关接口:

- 1) 8个拨档开关
- 2) 9个按键开关
- 3) 8个LED灯
- 4) 6个七段数码管

相关接口使用及管脚约束参考:

[走近 FPGA]之开发板介绍篇.pdf

开发软件

➤ 推荐vivado

➤ 下载和安装：见群文件

➤ 使用参考如下网址：

https://blog.csdn.net/m0_53606280/article/details/123596194

大规模数字电路/芯片设计流程

➤ 步骤1

确定芯片具备哪些功能、输入输出接口等

➤ 步骤2

模块划分、架构设计等

➤ 步骤3

模块描述（实现）

大规模数字电路/芯片设计流程

➤ 步骤4

功能仿真、行为仿真（前仿真，只是验证逻辑，未加入门延迟、线延迟等信息）

➤ 步骤5（可加约束）

综合1——翻译（逻辑库，门延迟为估算延迟）

此步骤后，可进行post-translate simulation

综合2——映射（器件库，门延迟为真实器件延迟）

此步骤后，可进行post-map simulation



大规模数字电路/芯片设计流程

➤ 步骤6

布局布线——将功能块合理的放置在芯片上，加上**I/O pad**，并进行连线。（可加约束）

该步骤后，可进行**post-route simulation**，此时，延迟信息（主要包括门延迟、线延迟、**I/O口延迟**等）估算最为真实

➤ 步骤7

FPGA验证

➤ 步骤8——芯片后端（形式验证、设计规则检查**DRC**等）、流片等



VerilogHDL

硬件描述语言介绍



目录

- 什么是Verilog HDL?
- Verilog HDL语法
- 设计描述层次
- 组合逻辑、时序逻辑、有限状态机
- 设计测试与验证
- 可综合的设计

Verilog HDL(1/2)

➤ Verilog HDL是一种硬件描述语言；
用于数字系统的设计

➤ 各种级别的逻辑设计：

算法级、结构级、门级到开关级等多种抽象设计层次的数字系统建模

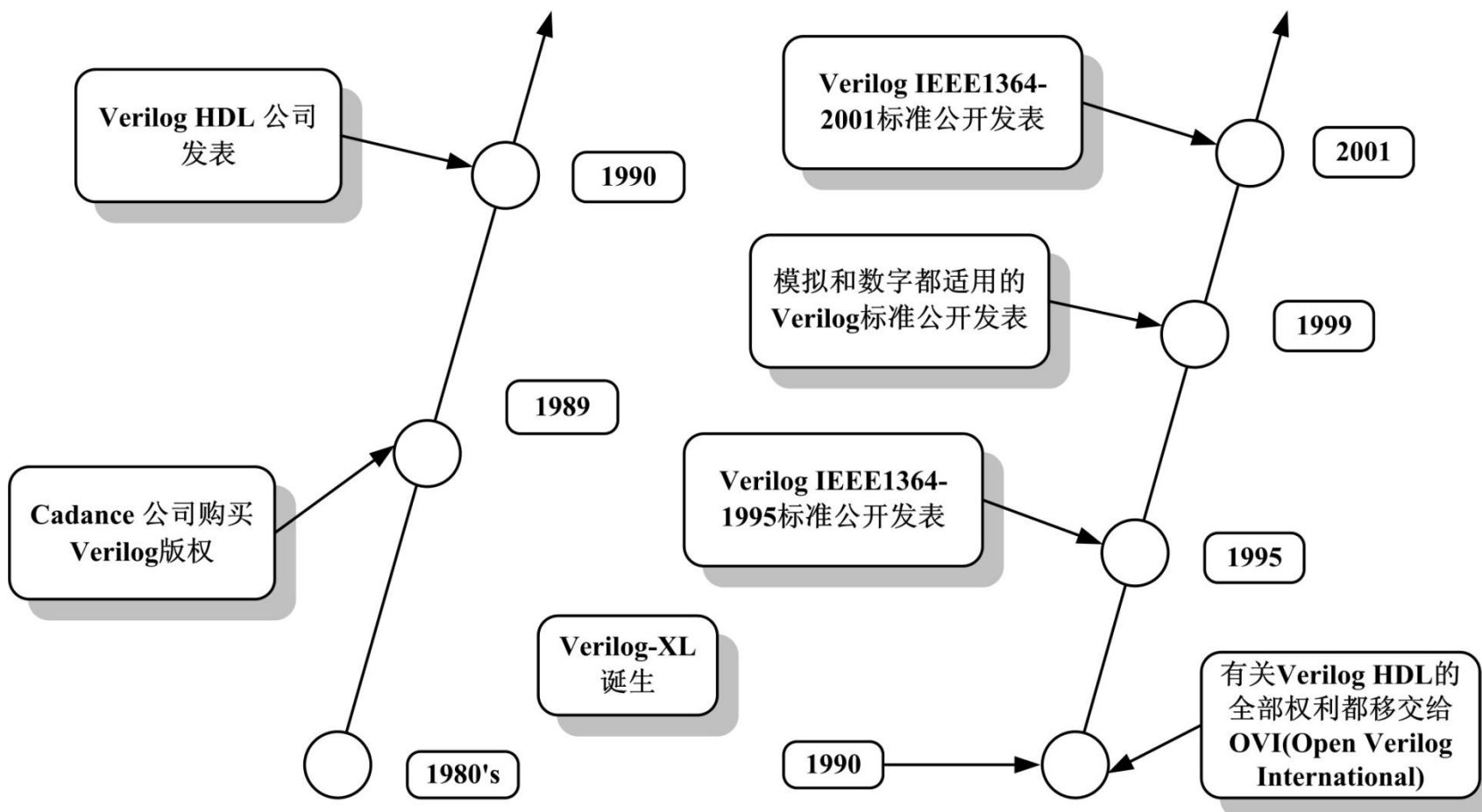
被建模的数字系统对象的复杂性可以介于开关级电路（如pmos/nmos）、简单的门和完整的复杂电子数字系统之间（例如CPU）

Verilog HDL(2/2)

- 进行数字逻辑系统的仿真验证、时序分析、逻辑综合
- 特点：
 - 从**C**语言继承了多种操作符和结构
 - 应用广泛
 - 比**VHDL**更简捷、灵活
 - 语法类似**C**，但思想不同，着眼于电路

Verilog HDL的历史

➤ 首创：1983, GDA (GateWay Design Automation)



Verilog HDL的结构

- 模块(**Module**)—可对应各种规模电路模块

module name(port_list);

Declarations: 端口、参数、信号等声明

parameter, input, output, inout, reg, wire,
function, task...

Statements: 语句，可以是功能描述语句，或其他
模块实例化调用

initial, always, assign,
module instantiation...

endmodule

➤ 一个例子:

检测1011序列的序列检测器, FSM实现

```
module seq_detect (clk,rst, data_in,
    seq_detect);
    input    clk, rst, data_in;
    output   seq_detect;
    parameter Init      =3'b000,
               Detect_1  =3'b001,
               Detect_10 =3'b010,
               Detect_101 =3'b011,
               Detect_1011 =3'b100;
    reg [2:0] state;
    assign seq_detect = (state==Detect_1011)?1'b1:1'b0;
```

端口参数声明

```
always@(posedge clk or posedge rst)
begin
    if(rst) state <= Init;
    else begin
        case(state)
            Init:
state <= data_in? Detect_1:Init;
            Detect_1:
state <= data_in? Detect_1:Detect_10;
            Detect_10:
state <= data_in? Detect_101:Init;
            Detect_101:
state <= data_in?
            Detect_1011:Detect_10;
            Detect_1011:
state <= data_in? Detect_1:Detect_10;
            default: state <= Init;
        endcase
    end
end
endmodule
```

功能描述



➤ 一个例子:

检测**1011**序列的序列
检测器，
移位寄存器实现

```
module mydff(clk, rst, d, q);  
input  clk, rst, d;  
output q;  
reg    q;  
always@(posedge clk or posedge rst)  
begin  
    if(rst)  
        q <= 1'b0;  
    else  
        q <= d;  
end  
endmodule
```

子模块定义

```
module seq_detect (clk,rst, data_in, seq_detect, sreg);  
input  clk, rst, data_in;  
output seq_detect;  
output [3:0] sreg;  
assign seq_detect = (sreg==4'b1011)?1'b1:1'b0;  
mydff d1(.clk(clk), .d(data_in), .rst(rst), .q(sreg[0]));  
mydff d2(.clk(clk), .d(sreg[0]), .rst(rst), .q(sreg[1]));  
mydff d3(.clk(clk), .d(sreg[1]), .rst(rst), .q(sreg[2]));  
mydff d4(.clk(clk), .d(sreg[2]), .rst(rst), .q(sreg[3]));  
endmodule
```

端口声明

子模块调用



目录

- 什么是Verilog HDL?
- Verilog HDL语法
- 设计描述层次
- 组合逻辑、时序逻辑、有限状态机
- 设计测试与验证
- 可综合的设计

Verilog HDL语法

- 模块
- 数据类型及常量、变量
- 运算符与表达式
- 条件与循环语句
- 结构说明语句

模块(module) (1/2)

➤ Verilog 基本设计单元:

一个电路模块/一种逻辑功能

基本元素: 接口, 逻辑功能

命名:

字母, 数字, \$, _

第一个必须为:

字母/_

区分大小写

```
module name(port_list);
```

Declarations:

parameter, input, output, inout, reg,
wire, function, task...

Statements:

initial, always, assign,
module instantiation...

```
endmodule
```


模块(module) (2/2)

➤ module 实例化

模块名称 实例化名称(.接口名称(信号名称),...);

若接口按顺序的排列可以省略接口名称

```
module adder(a,b,c);
```

```
input [1:0] a, b;
```

```
output[2:0] c;
```

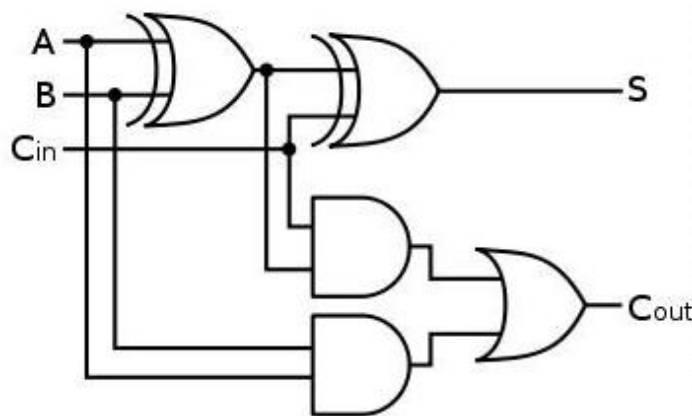
```
assign c = a + b;
```

```
endmodule
```

```
adder u1(.a(in1), .b(in2), .c(in3));
```

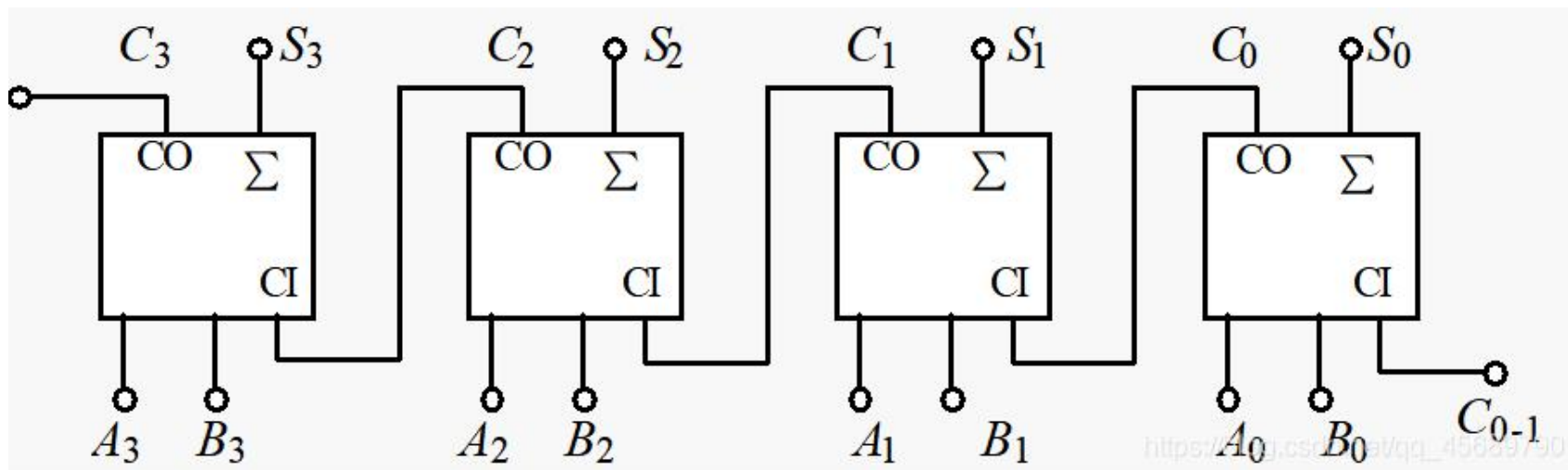
实例化语句使用练习

使用Verilog语言
内置的与门、或
门、异或门实现
一位全加器



```
assign sum = a^b^cin;
assign cout = (a&b)|((a^b)&cin);
```

使用上面设计的一位全加器实现逐次进位
（串行）加法器，并加入管教约束





数据类型及其常量变量(1/5)

➤ Verilog HDL有下列四种基本的值:

1) 0: 逻辑0或“假”

2) 1: 逻辑1或“真”

3) x: 未知

4) z: 高阻

➤ x, z值不区分大小写

➤ z 也可以用 ? 表示



数据类型及其常量变量(2/5)

- Verilog HDL中有三类常量:

1) 整型:

缺省十进制: 1, -2

基数表示: <位宽>'<进制><数字> b/B, o/O, d/D, h/H

4'b0101, 8'h42, -8'd5

2) 实数型: 1.234, -56.78, 3.6e4,

3) 字符串型: 双引号内, 每个字母用8位的ASCII值表示

如: “verilog!!” – 看作9个8位无符号整数,

反斜杠 (\) 用于定义特殊转义字符, 例如\n, \t

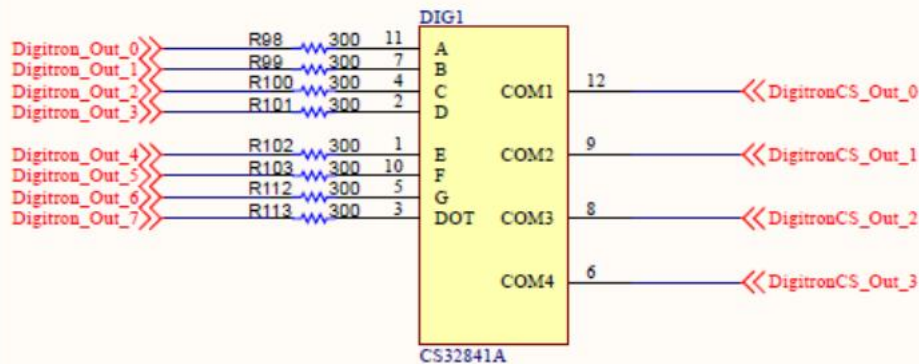
- 下划线 (_) 可以用来分隔数的表达式以提高易读性; 但不能用于位宽和进制处。例如:

32'b0010_1101_1110_1111 == 32'b0010110111101111

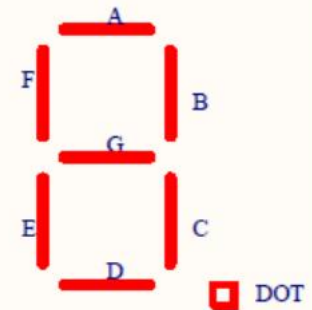
数据类型练习 (1)

- 请写出共阴八段数码管0~9各个数对应的二进制段码
 - SEG[7:0]为数码管的8个段码dot、g、f、e、d、c、b、a

四位动态共阴数码管



DIG



7段数码管说明



数据类型及其常量变量(3/5)

➤ Verilog HDL 变量最主要的数据类型:

- 1) 线网类型 (net type) : 表示Verilog结构化元件间的物理连线。它的值由驱动元件的值决定, 例如连续赋值或门的输出。如果没有驱动元件连接到线网, 线网的缺省值为z。常用: wire
- 2) 寄存器类型 (register type) : 表示一个抽象的数据存储单元 (但不表示它就一定综合成寄存器)。寄存器类型缺省值为x。常用: reg

Input和Output默认是wire类型

在always语句和initial语句中的赋值对象只能是reg类型, reg类型信号也只能在always语句和initial语句中被赋值, 所以, always、initial块外的赋值对象和连线用wire型信号, always、initial块内的赋值对象用reg型。



数据类型及其常量变量(4/5)

➤ 线网类型:

wire 声明: `wire [n-1:0] name1,name2,..., namei;`

➤ 举例:

`wire rdy, finish;` //2个1位的连线

`wire [3:0] result;` //1个4位的连线

➤ 可用作任何方程式或元件的输入, `assign`语句或者元件的输出
使用`assign`赋值

例如: `assign result = a + b;`

`assign`语句赋值对象的类型必须是`wire`型, 不可以是`reg`型。



数据类型及其常量变量(5/5)

➤ 寄存器类型:

reg 声明: `reg[n-1:0] name1, name2;`

➤ 举例:

`reg rdy, finish;` //2个1位的寄存器

`reg [3:0] result;` //1个4位的寄存器,可取任意一位: **result[2]**

`reg [3:0] mem [63:0];` //64个4位寄存器组,可表示存储器
mem[2], mem[2][1:0], //一个4位寄存器, 1个2位寄存器

数据类型练习 (2)

➤ 下面程序是否有问题？

```
module add_4bit_zc(data1,data2,sum);  
input [3:0] data1, data2;  
output [4:0] sum;  
  
assign sum= data1+data2;  
  
endmodule
```

```
module add_4bit_zc(data1,data2,sum);  
input [3:0] data1, data2;  
output [4:0] sum;  
  
always@(data1, data2)  
begin  
sum<= data1+data2;  
end  
  
endmodule
```

运算符与表达式(1/2)

- 算术运算符: (+, -, *, /, %)
- 赋值运算符: (=, <=)
- 关系运算符: (>, <, >=, <=)
- 相等运算符: (==, !=)
- 逻辑运算符: (&&, ||, !)
- 位运算符: (~, &, |, ^)
- 移位运算符: (<<, >>)
- 条件运算符: (?:) `assign result=exp ? exp1 : exp2;`
(若exp为真, 则result等于exp1, 否则exp2)
- 连接和复制操作符: ({}), `{exp1, exp2}, {rep{exp}}`
`assign bus[3:0] = { bus[0], bus[1], bus[2], bus[3]};`
`assign bus[3:0] = {2{bus[0]}, 2{bus[3]}};`



运算符与表达式(2/2)

➤ 表达式：操作数 + 运算符

➤ 分类：

单目运算符：带一个操作数 如：~clock

双目运算符：带两个操作数 如：a & b

三目运算符：带三个操作数 如：r = s ? a : b;

➤ 优先级：

(! ~) > (* / %) > (+ -) > (<< >>) > (< <= > >=)
> (== !=) > (&) > (^) > (|) > (&&) > (||)
> (?:)

块语句

- 两条或多条语句的组合，主要有两种：

begin_end语句：顺序块

块内的语句顺序执行；

每条语句的延时为相对于前一语句

最后一句执行完，才能跳出该块

```
begin: [name]  
    #n 语句1;  
    #m 语句2; .....  
end
```

fork_join语句：并行块

块内语句同时执行；

每条语句的延时为相对于进入块仿真的时间；
(较为少用)

```
fork: [name]  
    #n 语句1;  
    #m 语句2; .....  
join
```

条件语句

➤ **if else** 语句（需在**always**或者**initial**块中使用）

if(表达式) 语句;

[else if(表达式) 语句;]

[else 语句;]

多个语句需放在**begin end**间

➤ **case** 语句：多分支选择语句
（需在**always**或者**initial**块中使用）

case(表达式)

分支： 语句;

default: 语句;

endcase

```
if (a==b)
begin
    out1 <= in0;
    out2 <= in0;
end
else if (a>b)
begin
    out1 <= in1;
    out2 <= in2;
end
else
begin
    out1 <= in2;
    out2 <= in1;
end
```

若一个分支中需要多条语句，可以用
begin_end将多条语句变成一条块语句

循环语句(1/2)

- **forever** 连续执行，常用于产生时钟信号

如: ``timescale 1ns/100ps`

`forever #5 clk = ~clk; //放在initial块语句里，产生100M 的时钟`

- **while** 执行语句，直到某个条件不满足

`while (表达式)` 若开始即不满足条件，则不执行

`begin 语句; end`

- **repeat**

连续执行语句n次

`repeat(表达式)` 在此表达式通常为常量表达式，表示重复次数

`begin 语句; end`

循环语句(2/2)

➤ **for** **for**(表达式1; 表达式2; 表达式3) 语句;

例: **for**(i=0; i<4; i=i+1) a[i] <= b[i];

执行过程:

(1)求解表达式1

(2)求解表达式2, 若为真, 转到(3)

若为假, 结束循环, 转到(5)

(3)若表达式2为真, 执行语句后, 求解表达式3

(4)转回(2) 继续

(5)执行**for**语句后面的语句

循环语句不可综合, 一般只用于算法级描述或者测试模块中。循环语句不能单独使用, 需要放在**always**或者**initial**语句中。

结构说明语句

➤ initial

仿真开始时对各变量初始化
生成激励波形作为电路测试信号
只执行一次，
程序模块中可有多个，并行执行

```
initial  
begin  
    语句1;  
    语句2; .....  
end
```

➤ always always@(触发条件) begin 语句; end

若触发条件满足，则被执行，满足一次执行一次
可有多个always块，并行执行

在always块中赋值的对象只能是reg信号



阻塞赋值与非阻塞赋值

➤ 非阻塞赋值 $\leftarrow$ (如: $b \leftarrow a$)

块结束后才完成赋值操作

b的值并不是立刻改变

常用于沿触发的**always**块

➤ 阻塞赋值 $=$ (如: $b = a$)

赋值语句执行完后, 块才结束

b的值在赋值语句执行后立刻改变

常用于组合逻辑的赋值

设: $a=1, b=2$

$b \leftarrow a;$

$c \leftarrow b;$

得到: $c=2$

$b = a;$

$c = b;$

得到: $c=1$

练习

- 请设计一个**3-8译码器** 实验2
 - 输入：3比特二进制数000~111
 - 输出：8比特二进制数，例：000-→11111110, 001-→11111101, ...
- 请设计一个比较器，对输入的三个**3**比特二进制数进行比较，输出最大的**3**比特二进制数。 实验3



目录

- ❖ 什么是Verilog HDL?
- ❖ Verilog HDL语法
- ❖ 设计描述层次
- ❖ 组合逻辑、时序逻辑、有限状态机
- ❖ 设计测试与验证
- ❖ 可综合的设计

设计描述层次

- 算法（行为）级描述
- 寄存器传输级（**RTL**）描述
- 开关级（门级）描述
- 混合层次描述



算法（行为）级描述

- 主要用于快速验证算法的正确性
- 不一定可以综合成实际电路结构

实例：n位整数除法器, $D = A/B$, $R = A \% B$

```
module div (A, B, D, R);  
parameter n = 32;  
input  [n-1:0] A, B;  
output [n-1:0] D, R;  
reg    [n-1:0] D, R;  
always @(A or B)  
begin  
    D = 0;  
    for(R=A; R>B; R = R - B)  
    begin  
        D = D + 1;  
    end  
end  
endmodule
```

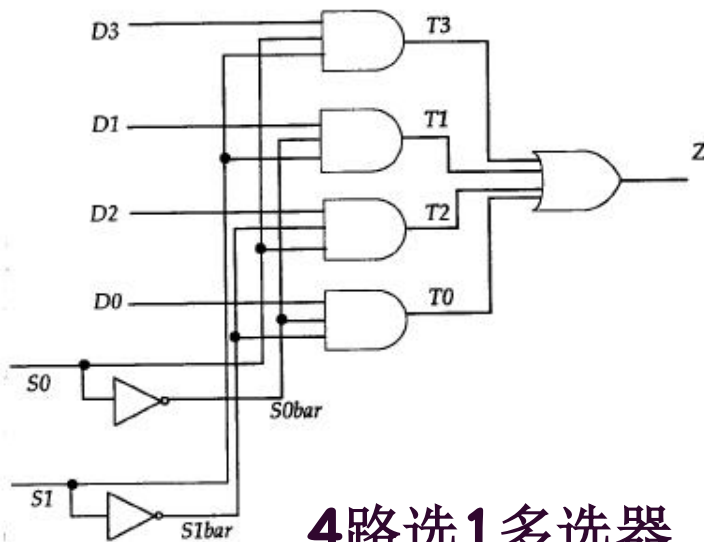
RTL描述

- 更贴近实际电路结构的描述
- 描述的细节到寄存器内容传输级别
- 可以精确描述电路的工作原理、执行顺序
- 细化到寄存器级别的结构描述也就是RTL级描述，并无绝对划分标准

```
module sample_reg(clk, rst, din, dout);  
input      clk, rst;  
input  [3:0] din;  
output [3:0] dout;  
reg  [3:0] data;  
assign dout = data;  
always @(posedge clk or negedge rst)  
begin  
    if(!rst) begin  
        data <= 4'd0;  
    end else begin  
        data <= din;  
    end  
end  
endmodule
```

开关级（门级）描述

- 完整描述了电路的细节
- 最底层的电路描述



4路选1多路器

```
module MUX4x1(Z, D0, D1, D2,
               D3, S0, S1);
    input  D0, D1, D2, D3, S0, S1;
    output Z;
    and    u0 (T0, D0, S0bar, S1bar) ,
           u1 (T1, D1, S0bar, S1),
           u2 (T2, D2, S0, S1bar),
           u3 (T3, D3, S0, S1) ;
    not    u4 (S0bar, S0),
           u5 (S1bar, S1);
    or     u6 (Z, T0, T1, T2, T3);
endmodule
```

混合层次描述

- 以上各种层次描述方式可以任意混合使用，因此一个设计可能某些模块使用了行为描述的方式，而另外一些模块使用了开关级描述
- 在实际设计中，功能复杂的模块可以先用行为级描述来表示，而对功能简单的模块则使用RTL级的描述，以尽早完成系统的仿真和算法的验证；在设计的过程中逐步细化复杂模块，替代原来的行为描述，直至设计可以综合并满足设计指标



目录

- ❖ 什么是Verilog HDL?
- ❖ Verilog HDL语法
- ❖ 设计描述层次
- ❖ 组合逻辑、时序逻辑、有限状态机
- ❖ 设计测试与验证
- ❖ 可综合的设计

组合逻辑电路

- 组合逻辑：输出只是当前输入逻辑电平的函数，与电路的原始状态无关的逻辑电路。
- 组合逻辑由与、或、非门组成的网络，如：多路器、比较器、加法器、乘法器等
- 描述方式：**assign**语句，阻塞赋值=，& | ~ 操作符

and, or, not门组合
always@电平触发

三种描述方式例子

```
module adder(a, b, cin, sum);  
input      a, b, cin;  
output [1:0] sum;  
wire      s, co;  
assign    s  = a ^ b ^ cin;  
assign    co = a & b | a & cin | b & cin;  
assign    sum = {co, s};  
endmodule
```

```
module add_4bit_zc(data1,data2,sum);  
input [3:0] data1, data2;  
output [4:0] sum;  
reg [4:0] sum;  
always@(data1, data2)  
begin  
sum<= data1+data2;  
end  
endmodule
```

```
module adder(a, b, cin, sum);  
input      a, b, cin;  
output [1:0] sum;  
wire s, co;  
wire tmp0,tmp1,tmp2,tmp3  
assign    sum = {co, s};  
xor  x1(tmp0,a,b),  
      x2(s,tmp0,cin);  
nand  x3(co,tmp1,tmp2,tmp3),  
      x4(tmp1,a,b),  
      x5(tmp2,a,cin),  
      x6(tmp3,b,cin);  
endmodule
```



组合逻辑电路练习

- 设计一个带数字显示的**3-8**译码器 实验4
- 设计一个加减法器，根据运算选择开关，对两个**3**比特数进行加法或减法运算，结果十位显示在**LED**上，个位显示在数码管上。 实验5

时序逻辑电路

- 时序逻辑：输出不只是当前输入的逻辑电平的函数，还与电路目前所处的状态有关的逻辑电路
- 由多个触发器和多个组合逻辑块组成，如：计数器、同步有限状态机、移位寄存器等
- 描述方式：

`always@(posedge clk)` 非阻塞赋值 `<=`

触发器和组合电路连接

时序逻辑电路一定要有复位信号，而且此信号一般只用于复位。实际应用中，电路板一般都有上电复位设计



时序 逻辑 电路 描述 例子

```

module couter10(reset, clk, dout);
input reset,clk;
output[3:0] dout;
reg [3:0] dout;
reg [25:0] clk_t;
reg clk_i;
always @ (posedge clk )begin
if (!reset)
begin
clk_t<=0;
clk_i<=1;
end
//else if (clk_t==26'b10011000100101101000000000) //5.341ns
//else if (clk_t==26'b1000000000000000000000000000) //5.156ns
else if (clk_t[25]==1'b1) //4.701ns
begin
clk_t<=0;
clk_i<=1;
end
else
begin
clk_t<=clk_t+1;
clk_i<=0;
end
end
always @ (posedge clk_i)
Begin
if (!reset)
dout<=0;
else if (dout==4'd9)
dout<=0;
else
dout<=dout+1;
end
endmodule

```

设计注意事项

➤ 注意区分同步复位和异步复位

```
always@(posedge clk or posedge rst)
begin
    if(rst)
        cnt <= 4'd0;
    else
        cnt <= cnt + 1'b1;
end
```

异步复位

```
always@(posedge clk or negedge rst)
begin
    if(!rst)
        cnt <= 4'd0;
    else
        cnt <= cnt + 1'b1;
end
```

异步复位

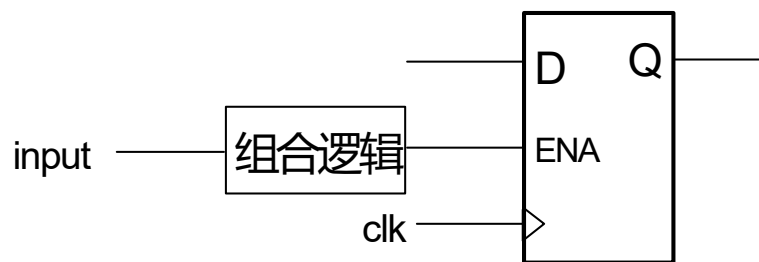
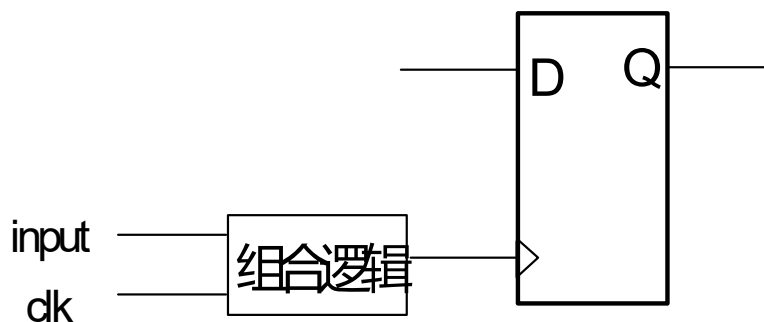
```
always@(posedge clk)
begin
    if(rst)
        cnt <= 4'd0;
    else
        cnt <= cnt + 1'b1;
end
```

同步复位

设计注意事项

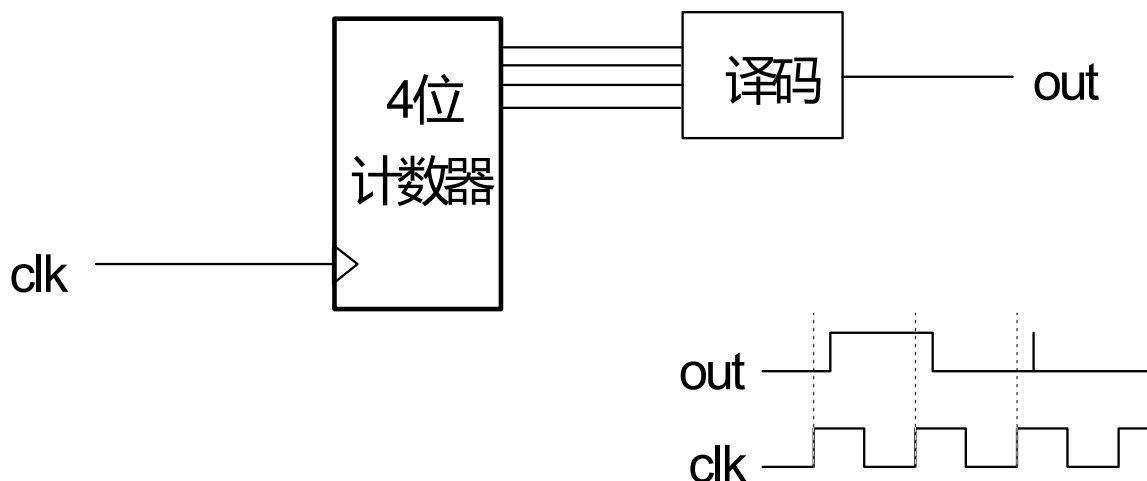
❖ 避免对时钟进行逻辑操作后作为触发信号

不建议使用门控时钟和组合逻辑时钟，易产生毛刺，发生误动作，建议使用触发器的使能端，不增加资源



设计注意事项

❖ 慎重对计数器进行译码



可能由于竞争冒险产生毛刺。如果后级采用了同步电路，我们完全可以对此不予理会。如果对毛刺要求较高，推荐采用Gray编码(PLD)或One-hot编码(FPGA)的计数器，一般不要采用二进制码。

设计注意事项

➤ 避免多驱动问题

电路块1
(always/assign)

电路块2
(always/assign)

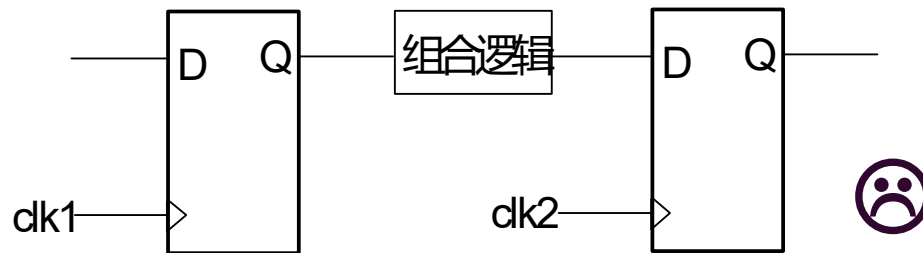
驱动同一个
信号，即同
时对一个信
号进行赋值

这是错的

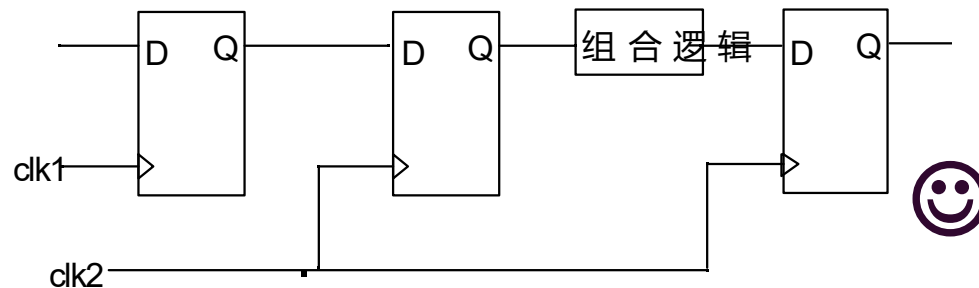
设计注意事项

❖ 多时钟的同步化

两个没有相位关系的时钟这样使用，后级时钟可能不能准确采样



改进后：用一个触发器做同步化处理，保证后级时钟准确采样。



设计注意事项

- 避免C语言描述方式（虽然C语言vivado也能综合成电路，但这背离了本课程的学习初衷，而且，综合出来的电路也不可控）

```
always @( * ) module binbcd(          z[10:3]=b;
begin          input [7:0] b,          repeat(5)
    an = 4'b1111; output reg[7:0] p begin
    if(clr==1) );          if(z[11:8]>4)
        an[s] = 0; reg[17:0] z;          z[11:8]=z[11:8]+3;
end          integer i;          if(z[15:12]>4)
                always@(*)          z[15:12]=z[15:12]+3;
                begin          z[17:1]=z[16:0];
                    for(i=0;i<=17;i=i+1) end
                        z[i]=0;          p=z[15:8];
                                end
                                endmodule
```

时序逻辑电路练习

- 按键防抖实验（用按键作为计数器的时钟输入，可以实现每按一下按键计数器加1） 实验6
- 采用动态扫描方式在4个八段数码管上分别显示0~3 实验7
- 设计一个秒表，8段数码管显示。显示示意图如下：



实验8

- 设计一个加减法器，根据运算选择开关，对两个4比特数进行加法或减法运算，结果用十进制方式显示在在数码管上。 实验9



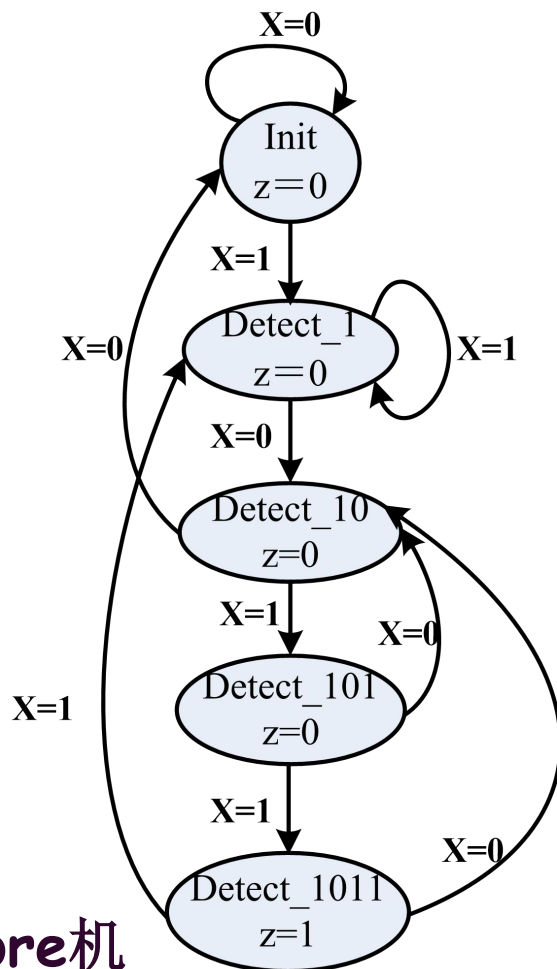
有限状态机(FSM) (1/4)

- 状态机是绝大部分控制电路的核心结构
- 有限状态机可以使用**always**语句和**case**语句描述。将状态保存于寄存器中，根据寄存器中不同的取值（状态）执行不同的操作，**case**语句的多个分支则代表了不同状态的行为
- **Moore**有限状态机：输出仅依赖于内部状态，跟输入无关
- **Mealy**有限状态机：输出不仅决定于内部状态，还跟外部输入有关

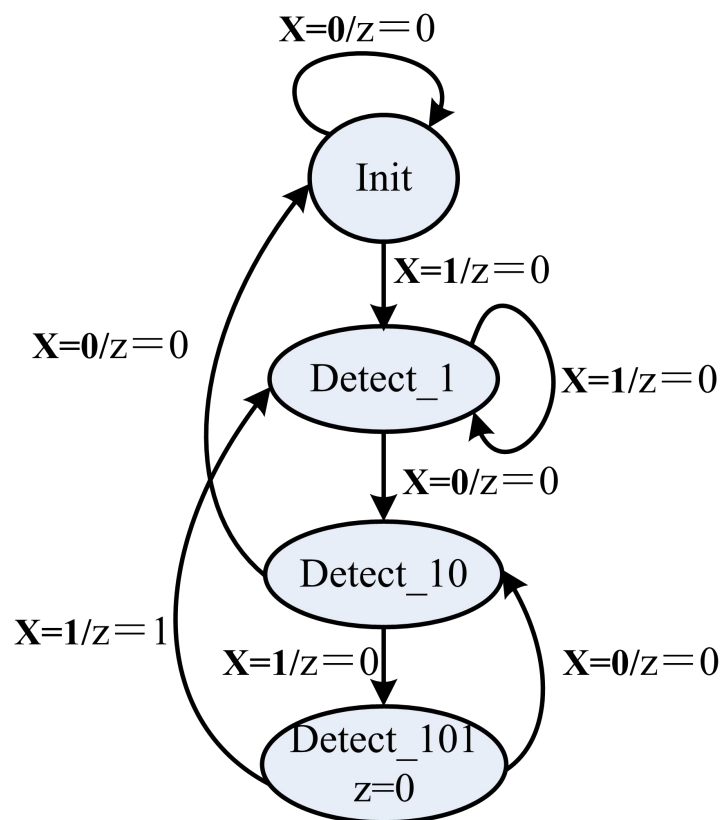


有限状态机(FSM) (2/4)

➤ 1011序列检测器，可重复检测



Moore机



Mealy机



有限状态机(3/4)

```

module seq_detect (clk,rst, data_in,
    seq_detect);
input    clk, rst, data_in;
output   seq_detect;
parameter Init      =3'b000,
           Detect_1  =3'b001,
           Detect_10 =3'b010,
           Detect_101=3'b011,
           Detect_1011=3'b100;
reg [2:0] state;
assign seq_detect =
    (state==Detect_1011)?1'b1:1'b0;

```

Moore机描述

```

always@(posedge clk or posedge rst)
begin
    if(rst) state <= Init;
    else begin
        case(state)
            Init:
state <= data_in? Detect_1:Init;
            Detect_1:
state <= data_in? Detect_1:Detect_10;
            Detect_10:
state <= data_in? Detect_101:Init;
            Detect_101:
state <= data_in?
            Detect_1011:Detect_10;
            Detect_1011:
state <= data_in? Detect_1:Detect_10;
            default: state <= Init;
        endcase
    end
end
endmodule

```


有限状态机(4/4)



```
module seq_detect (clk,rst,
    data_in, seq_detect);
input    clk, rst, data_in;
output   seq_detect;
parameter Init    =3'b000,
           Detect_1 =3'b001,
           Detect_10 =3'b010,
           Detect_101 =3'b011;
reg [2:0] cstate, nstate;
assign seq_detect = data_in
    &&(cstate==Detect_101);
always@(posedge clk or posedge
    rst)
begin
    if(rst)    cstate <= Init;
    else      cstate <= nstate;
end
```

```
always@(cstate or data_in)
    begin
case(cstate)
    Init:
nstate = data_in? Detect_1:Init;
    Detect_1:
nstate = data_in?Detect_1:Detect_10;
    Detect_10:
nstate = data_in? Detect_101:Init;
    Detect_101:
nstate = data_in?Detect_1:Detect_10;
    default:
nstate = Init;
endcase
end
endmodule
```

Mealy机描述



目录

- ❖ 什么是Verilog HDL?
- ❖ Verilog HDL语法
- ❖ 设计描述层次
- ❖ 组合逻辑、时序逻辑、有限状态机
- ❖ 设计测试与验证
- ❖ 可综合的设计

设计测试与验证

- 一般需要编写测试文件对设计进行仿真测试称为**testbench**
- **testbench**一般采用行为级描述，产生一定的输入激励，得到设计的输出响应，并检查输出结果的正确性
- 在规范的模块设计中，每一个模块都需要编写独立的测试文件对模块进行充分的测试
- 注意选择合适的输入激励，保证测试能够覆盖典型输入和特殊输入

设计测试与验证

➤ 计数器例子

```
module counter (clk, rst, cnt);  
input    clk, rst;  
output[3:0] cnt;  
reg[3:0] cnt;  
always@(posedge clk or posedge rst)  
begin  
    if(rst)  
        cnt <= 4'd0;  
    else  
        cnt <= cnt + 1'b1;  
    end  
endmodule
```

```
module test;  
reg    clk, rst;  
wire [3:0] cnt;  
counter U1 (clk, rst, cnt);  
counter U1(.clk(clk), .rst(rst),  
           .cnt(cnt));  
initial  
begin  
    clk = 1'b0;  
    rst = 1'b0;  
    #10 rst = 1'b1;  
    #40 rst = 1'b0;  
end  
always # 20 clk = ~clk;  
endmodule
```

*Modelsim*仿真有
时会出问题

好的表
达方式

系统函数和任务

- Verilog 提供一些封装的系统任务或函数，以标识符\$ 确认

\$display, \$monitor

\$time, \$realtime, \$printtimescale

\$stop, \$finish

\$fopen, \$fclose

\$fdisplay, \$fwrite

\$readmemb, \$readmemh

\$random

- 例如：

\$display ("hello world!"); //打印输出hello world

\$time //该系统任务返回当前的模拟时间。



目录

- ❖ 什么是Verilog HDL?
- ❖ Verilog HDL语法
- ❖ 设计描述层次
- ❖ 组合逻辑、时序逻辑、有限状态机
- ❖ 设计测试与验证
- ❖ 可综合的设计

可综合的设计

- 可综合：由综合工具将设计转换为门级电路
- 可综合成电路的只是**Verilog**语法的一个子集
- 可综合的基本原则：
 - ▶ 每部分代码都映射成相应的电路
 - ▶ 有经验的工程师设计代码时已经知道基本电路结构
 - ▶ 人往往比综合器更聪明☺

可综合的语法举例

- **and**, **or**, **nand**等基本逻辑门
- **if**块和**case**块
- **+**, **-**, **&**, **|**, **^**, **~**等逻辑运算符
- **&&**, **||**, **==**, **>**, **>=**, **<**, **<=**, **!=**, **!**等关系运算符
- **>>**, **<<**, **&**（按位与）, **|**（按位或）, **{ }**（连接运算）等位操作运算符
- **=**, **<=**, **assign**等赋值语句
- **always**块



不可综合的语法举例

- **initial**块（综合时会被忽略）
- **for, while, repeat, forever, force, release, task, enable, disable**一般不可综合
(注：某些特定情况下**for**、**while**、**repeat**等可能可以综合,但综合效率无法保证，不建议使用)
- 系统函数如**\$display**等
- 除法，求模运算，实数运算等复杂算术运算一般不可综合（注：若综合器有库可以引用，则可以综合）
- **#**延时赋值语句一般综合时会被忽略
-

可综合的设计

- **case**语句一般对应多选器

分支要完全，**default**分支的使用

赋值为“**x**”为无关项，综合结果最简捷

- 避免出现**Latch**

生成组合逻辑的**always**块中，赋值表达式右边出现的信号都必须在敏感列表中列出

- 异步状态机不能综合

综合器只支持同步系统的设计

内容回顾

- Verilog HDL的语法
- 设计的描述层次
- 简单组合逻辑电路、时序逻辑电路和有限状态机的实现
- 可综合的设计
- 仅介绍了语言的基本使用，代码风格会影响综合得到电路的性能，优化的设计还需要经验的积累☺

实验内容推荐

- 基本组合逻辑电路设计
 - 用移位加3法实现4比特Bin转BCD

Shift and Add-3 Algorithm

1. Shift the binary number left one bit.
2. If 8 shifts have taken place, the BCD number is in the *Hundreds*, *Tens*, and *Units* column.
3. If the binary value in any of the BCD columns is 5 or greater, add 3 to that value in that BCD column.
4. Go to 1.

Operation	Tens	Units	Binary
HEX			E
Start			1 1 1 0
Shift 1		1	1 1 0
Shift 2		1 1	1 0
Shift 3		1 1 1	0
Add 3		1 0 1 0	0
Shift 4	1	0 1 0 0	
BCD	1	4	

实验内容推荐

- 基本组合逻辑电路设计
 - 用移位加3法实现8比特Bin转BCD

Operation	Hundreds	Tens	Units	Binary	
HEX				F	F
Start				1 1 1 1	1 1 1 1
Shift 1			1	1 1 1 1	1 1 1
Shift 2			1 1	1 1 1 1	1 1
Shift 3			1 1 1	1 1 1 1	1
Add 3			1 0 1 0	1 1 1 1	1
Shift 4		1	0 1 0 1	1 1 1 1	
Add 3		1	1 0 0 0	1 1 1 1	
Shift 5		1 1	0 0 0 1	1 1 1	
Shift 6		1 1 0	0 0 1 1	1 1	
Add 3		1 0 0 1	0 0 1 1	1 1	
Shift 7	1	0 0 1 0	0 1 1 1	1	
Add 3	1	0 0 1 0	1 0 1 0	1	
Shift 8	1 0	0 1 0 1	0 1 0 1		
BCD	2	5	5		

实验内容推荐

- 基本组合逻辑电路设计
 - 乘法器（4位二进制乘以4位二进制）

实验内容推荐

➤ 基本时序逻辑电路

- 分频器设计（占空比50%的七分频）
- 伪随机码生成和检测
- 用移位加三法实现数值范围在0~99999的可变位宽二进制转BCD码，例如：输入二进制位宽设置为12bit时，输入“101000000111”输出“0000 0010 0101 0110 0111”；输入二进制位宽设置为17bit时，输入“1100001101001111”输出“1001 1001 1001 1001 1001”（该题目也可以用有限状态机实现）
- FIFO(First Input First Output)存储器（最好采用RAM IP来存储数据，通过Block RAM generator工具生成）

➤ 有限状态机设计

- 自动售货机设计
- 交通灯设计

实验内容推荐

➤ 简单数字系统设计

- **乒乓球游戏设计**（供参考：8个LED灯视为球桌，未开球时LED全灭，开球之后任何时刻只有1个LED灯亮视为球的位置，按键作为球拍，按下时即挥拍，过网接才有效，可以设置1-2个接球位置，只有球到了规定位置且接球球员按下该位置对应的按键才算接球成功，早接或者晚接都视为接球失败。4个数码管显示每局得分，11分制；2个数码管显示大局得分，7局4胜。初始化时，LED灯全灭，只有一方挥拍后球才相应方起始位置开始移动）（注：按键按下为“0”）
- **计算加减运算训练系统**（利用伪随机数随机产生两个两位十进制数和加减运算符，得到随机计算式子显示在数码管上，比如XX-XX=，等待训练者回答后，按键按下后数码管上显示计算结果）

实验内容推荐

➤ 简单数字系统设计

- **专注度训练小游戏**（九个按键代表1-9九个数字，游戏开始后，开始计时（精度0.1秒，显示在三个数码管上），同时，在一个数码管上随机依次显示1-9九个数字，游戏者看到显示数字按下相应的按键，若按错键一次扣0.1秒时间（所扣总时间同时显示在两个数码管上），按正确则显示下一个数字等待用户按键，九个数字依次显示完成后，数码管上显示所用时间+被扣时间的总和）
- **抢答器设计、呼叫系统设计、密码锁电路设计、定时控制电路设计、频率计设计等**



Xiamen University

The End

Thanks!